

ReconMind

An agentic copilot for the part of data engineering I have spent years doing by hand:
working out why a pipeline's numbers are wrong, and writing it up.



Hybrid RAG + LangGraph multi-agent + MCP tools + LangFuse observability

GitHub github.com/rahulramachandran-labs/reconmind

Live reconmind-labs.vercel.app

SUBMITTED BY

Rahul

SUBMISSION DATE

18 September 2026

CONTENTS

What I will walk through



01

Problem statement & objectives



03

Solution & system architecture



05

Retrieval layer design



07

MCP tool servers



09

Application screens & tech stack



02

Course concepts applied



04

Synthetic dataset & injected anomalies



06

Multi-agent design (LangGraph)



08

Observability & evaluation



10

Build plan, learnings, future work

The problem I keep running into

After 13 years of running multi-source pipelines, the same four failures keep showing up. Someone senior spots them, digs through five systems, and the reasoning ends up in a chat thread nobody can find later.



Key drift

Same store, two IDs. One feed calls it outlet_id, another location_id, and the join quietly splits it in two.



Duplicate submissions

A vendor resends a file. The same transaction is now in twice, and the rule "latest file wins" exists only in someone's head.



Schema drift

Upstream renames a column on a Friday. Downstream finds out on Monday morning.



Volume anomalies

A feed comes in 40% light. Nobody notices until a dashboard looks off and someone asks why.

What I have tried before: dashboards tell you something is off but not why. A chatbot over the runbooks can explain an error but cannot go look at the pipeline. One big agent with every job in its prompt was impossible to debug.

What I wanted it to do

OBJECTIVES

- Catch all four failure types on a schedule, or whenever I ask it a question.
- Investigate with specialist agents that read live pipeline metadata through tools, not hardcoded API clients.
- Ground every conclusion in the runbooks, schema docs, and old incident write-ups I would normally pull up myself.
- Hand me a structured incident report: what broke, how many records, the likely root cause, a suggested fix.
- When it is not sure, stop and ask me instead of guessing.
- Trace every LLM and tool call so I can audit any answer after the fact.
- Keep the agents domain-agnostic so I can point the same system at a different problem later.



Scope & constraints

- Synthetic data only. A seeded generator mimics the shape of real reconciliation problems; nothing from any employer or client.
- Zero-cost demo mode: local Ollama plus sentence-transformers behind a provider fallback chain.
- Solo build, in phases, with tests gating each phase before the next starts.
- Not in scope: streaming ingestion, multi-tenant auth, production SLAs.

Where each course module shows up in the build

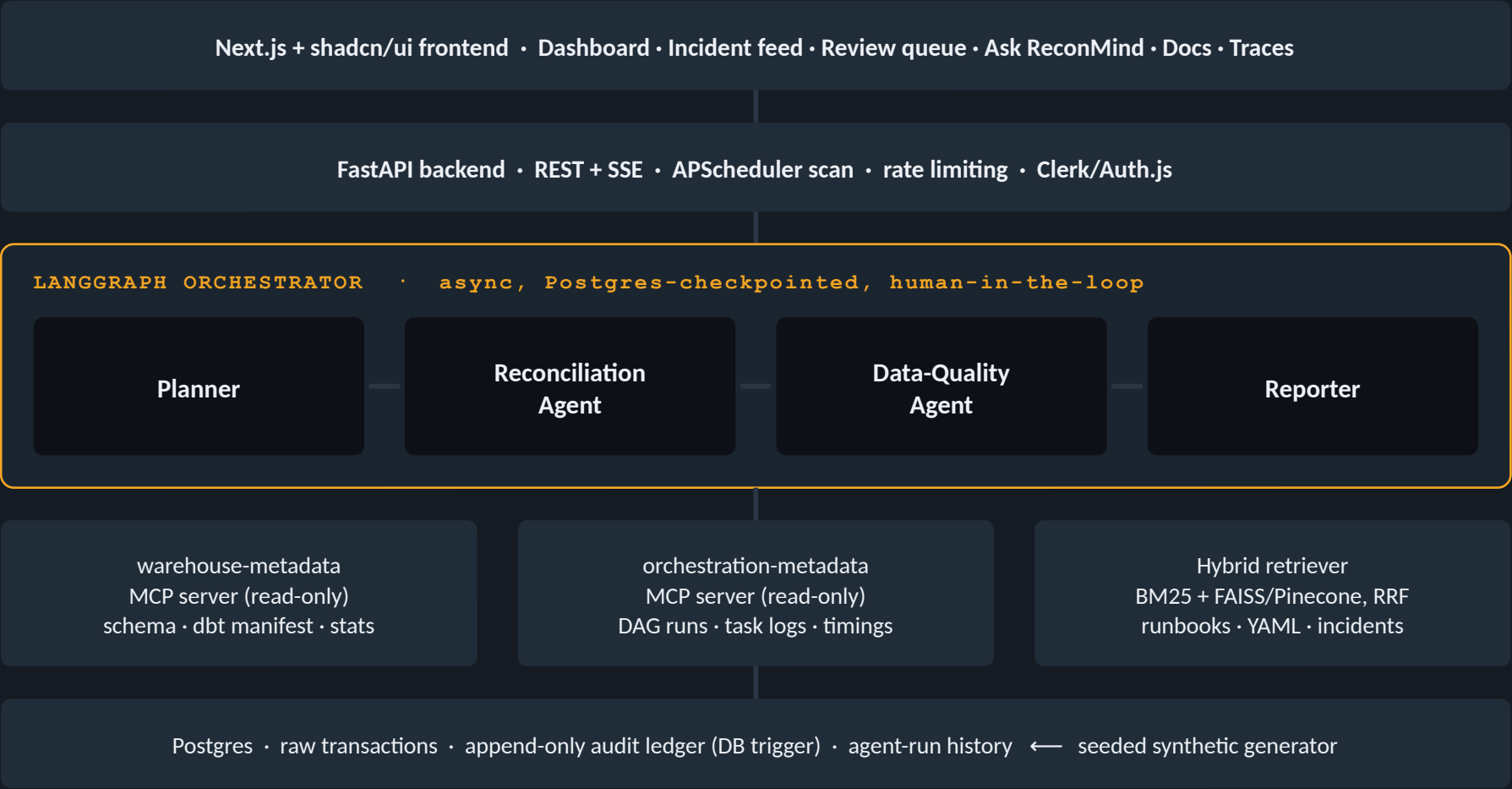
Module	Concept	How I used it
GenAI foundations	Prompting, structured output, context windows	Agent prompts with delimited untrusted context; Pydantic-validated outputs with retry loop
Models & APIs	Provider abstraction, fallback, cost awareness	OpenAI → Anthropic → Ollama fallback chain; provider and token cost logged per call
LangChain	Loaders, splitters, retrievers, chains, memory	Document ingestion of runbooks/YAML; session memory persisted to Postgres
RAG systems	Chunking, BM25 + dense hybrid, RRF, RAGAS evaluation	Hybrid retriever over FAISS/Pinecone; golden set with RAGAS gate in CI
Agentic AI	LangGraph StateGraph, conditional edges, checkpoints, multi-agent	Planner → Reconciliation Data-Quality → Reporter; human-in-the-loop checkpoint
MCP	Host/client/server, JSON-RPC lifecycle, stdio vs HTTP	Two read-only MCP servers: warehouse-metadata and orchestration-metadata
Observability & deployment	LangFuse tracing, FastAPI, Docker, Vercel/Render	Every step traced; containerized stack; public deployment with CI

How it fits together



Put simply: a question or a scheduled scan enters the graph. The Planner decides who needs to look. The specialists pull evidence through MCP tools and retrieved docs. The Reporter writes the incident report. LangFuse keeps the record of how it got there.

Component view



LangFuse

Every agent step, tool call, prompt version, latency, and token cost. Each incident links straight to its trace.



LLM providers

Runs fully locally with Ollama, or on the OpenAI API. Embeddings: OpenAI or local sentence-transformers. If no LLM is available at all, it still returns the most relevant passages with citations.

The data: a seeded generator with four bugs planted on purpose

TRANSACTIONS TABLE

```
outlet_id · location_id · transaction_id
upc_code · channel_basket_id
submitter_file_name · qty · amount · event_ts
```

FILE NAMING

```
SUBMITTERID_YYYYMMDD_HHMM_NAME.txt
```

ALSO GENERATED

- dbt-style model YAML + fake manifest.json
- Airflow-style DAG run log (JSON) with one failed task
- DATA_DICTIONARY.md documenting every column and anomaly
- --seed 42 default → byte-for-byte reproducible; sample under data/sample/



Key drift

~3% of rows: one location reporting under two outlet_ids



Duplicates

Same transaction_id + channel_basket_id + upc_code in two files; the later file should win



Schema drift

One batch drops or renames a column



Volume anomaly

One day's file comes in 40% under trailing average

Retrieval: why I use BM25 and embeddings together



1 Ingest

Runbooks, schema docs, dbt YAML, and past incident write-ups, chunked with overlap. The chunking choice is written up as an ADR.



2 Retrieve twice

BM25 nails exact table names, column names, and error codes. Dense search (FAISS locally, Pinecone behind a flag) handles the "why would this happen" questions.



3 Fuse

Reciprocal rank fusion merges the two lists so neither one's blind spot wins.



4 Guard

Retrieved text is untrusted. It is wrapped in delimiters, and a test plants an instruction inside a runbook to make sure the agents ignore it.

HOW I KNOW IT WORKS

I wrote a golden set of 40+ question / answer / context triples covering every anomaly type. RAGAS scores faithfulness, answer relevancy, context precision, and context recall on every commit, and CI fails below the thresholds in evals/thresholds.yaml (0.75 to start, raised in the last phase). Phase B has to beat Phase A on this set before I add any agents.

The agents: four narrow jobs instead of one big prompt



Planner

Takes the question or scan trigger and decides which specialists to run. If confidence is low, it routes to me instead of guessing.



Reconciliation

Applies the dedup keys and latest-file-wins rule from the domain adapter. Flags key drift as its own finding type.



Data-Quality

Diff's live schema against the dbt manifest and checks volume and timing against DAG history, all through MCP tools.



Reporter

Turns the findings into a Pydantic IncidentReport: problem, affected records, root-cause hypothesis, fix, confidence, open questions, trace link.

CONCURRENCY

Reconciliation and Data-Quality run in parallel with `asyncio.gather` when the Planner picks both. No reason to serialize work that does not depend on itself.

HUMAN-IN-THE-LOOP

Postgres-backed checkpointing pauses the graph. The Review queue lets me approve, reject, or annotate, and the graph picks back up.

VALIDATED OUTPUT

Every node returns a Pydantic model. A validation error re-prompts, capped at two retries.

Tool access: two read-only MCP servers



warehouse-metadata-server

Serves the dbt-style manifest and table stats so the Data-Quality agent can diff what was declared against what actually landed.

TOOLS EXPOSED

```
get_table_schema(table)
get_dbt_manifest()
get_table_stats(table, date)
list_tables()
```



orchestration-metadata-server

Serves Airflow-style run history so the agents can tie a light file to a late or failed upstream task.

TOOLS EXPOSED

```
list_dag_runs(dag_id, since)
get_task_log(run_id, task_id)
get_timing_history(dag_id)
get_failed_tasks(since)
```

PROTOCOL

JSON-RPC 2.0 · initialize → discover →
invoke lifecycle

TRANSPORT

stdio for local dev · HTTP for
containerized prod

SAFETY

Read-only tools · Pydantic-validated
inputs · no write paths

REUSE

Point both servers at a different backend
and the agents do not change

Observability and evaluation: how I trust what it says



LangFuse tracing

- Every agent call, tool call, prompt version, latency, provider, and token cost
- Each incident deep-links to its trace; a /traces screen lists recent runs
- Definition of done: no untraced LLM calls, anywhere



RAGAS evaluation

- 40+ golden-set items across all anomaly types
- Faithfulness, answer relevancy, context precision, context recall
- CI fails below threshold; scores logged per commit



Chaos & safety tests

- Every planted anomaly must be caught and typed correctly
- Prompt-injection test against a poisoned runbook
- Full investigation under 30 s in demo mode

WHAT I AM HOLDING IT TO

RAGAS $\geq$ 0.75 (raised in D)

All injected anomalies caught

Test coverage $\geq$ 80%

Investigation $<$ 30 s

Zero untraced LLM calls

The app: six screens, one graph behind all of them



Dashboard

Today's volume vs trailing average, open findings by severity, last DAG run, and what today cost in tokens.



Incident feed

Every finding, expandable into the full write-up, with a link straight to its LangFuse trace.



Review queue

Findings the graph paused on. Approve, reject, or annotate, and it picks back up.



Ask ReconMind

Streaming chat with session memory in Postgres. Runs the full graph on whatever I ask it.



Docs & runbooks

The ingested corpus, searchable on its own, so I can prove retrieval works before the agents touch it.



Traces

Recent runs with cost and latency, linking into LangFuse.

What I built it with

FRONTEND

Next.js (App Router)

shadcn/ui

Tailwind

Vercel

BACKEND

FastAPI

SQLAlchemy + Alembic

APScheduler

Render / Railway

ORCHESTRATION

LangGraph

LangChain

Pydantic

asyncio

RETRIEVAL

BM25 (rank_bm25)

FAISS → Pinecone

sentence-transformers

RRF fusion

TOOLS & MODELS

MCP (2 servers)

OpenAI / Anthropic

Ollama (local)

Provider fallback chain

OPS & QUALITY

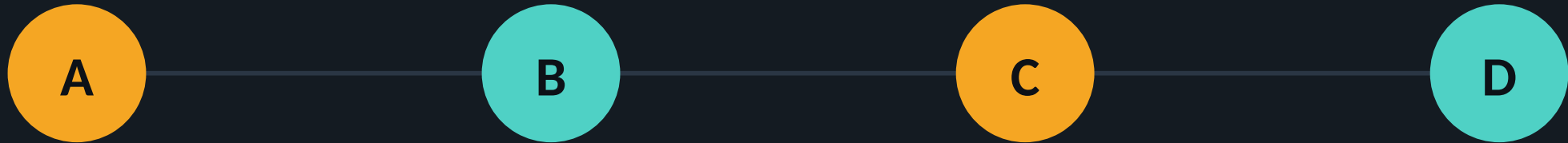
LangFuse

RAGAS

pytest + chaos suite

Docker Compose
GitHub Actions

How I am building it: four phases, each shipped before the next



Phase A • MVP

Repo + CI green, single LLM + basic RAG over synthetic docs, bare FastAPI + Next.js page, public URL.

Phase B • Production RAG

Seeded generator, hybrid retrieval + RRF, session memory, Postgres + ledger, provider fallback, Docker Compose, RAGAS gate.

Phase C • Multi-agent

Two MCP servers, LangGraph with four agents, domain adapter + stub, LangFuse, human-in-the-loop, injection test, live screens.

Phase D • Polish & deploy

Auth, scheduler, chaos suite, rate limiting, dashboard, raised eval thresholds, ADRs, demo GIF, blog draft, v1.0.0 release.

Rules I set for myself: conventional commits; pre-commit with ruff, mypy, and gitleaks; GitHub Actions running lint, tests (80% coverage floor), the RAGAS gate, and the frontend build on every push; a tag at each phase boundary; and a Decision Council review before I write a line of code for each phase.

What I learned designing this



One agent vs many

My first sketch was one agent with every job. The prompt was enormous, and when it failed I could not tell which part had failed. Four narrow agents fixed both problems.



Embeddings blur identifiers

Dense search kept confusing tables with similar names. Adding BM25 and fusing with RRF fixed the exact-match questions without losing the conceptual ones.



Retrieved text is untrusted input

Runbooks go straight into prompts. Once I thought about it that way, delimiting them and testing against a planted instruction stopped being optional.



Observability is the trust layer

If I cannot answer "why did it say that" without re-running everything, I will not trust it in production. A trace link on every finding is what makes it auditable.



Reusability must be provable

Saying it is domain-agnostic is easy. Moving every business rule into a DomainAdapter and booting the graph on a stub adapter in CI is what makes it true.

Where else I could point this

Swap the data generator and the two MCP servers. The retrieval layer, the agent graph, the tracing, and the UI shell stay exactly as they are.



HEALTHCARE

Claims Copilot

Claims-vs-EOB matching; coding and billing drift.



SAAS

Support Triage Copilot

Route tickets; check account state and known incidents; draft the response.



TRADING

Signal Review Copilot

Feed/tick anomalies; position and ledger drift across venues.

WHAT I WOULD DO NEXT

- Kafka ingestion so scans run when data lands, not on a timer.
- Let the Reporter open a pull request with the fix, gated on my approval.
- Tune the Planner's routing threshold from my own approve / reject history.
- Multi-tenant auth and an adapter registry so several domains can run side by side.



Conclusion

ReconMind helps engineers investigate data-pipeline incidents such as key drift, duplicate submissions, schema drift and volume anomalies. It uses RAG to search runbooks, schema docs and past incident reports, then answers with citations. It can run fully locally with Ollama or use the OpenAI API. If no LLM is available, it still returns the most relevant passages from those documents.

Grounded

Hybrid RAG with a RAGAS gate

Orchestrated

Four LangGraph specialists + HITL

Connected

Two read-only MCP servers

Auditable

Every step traced in LangFuse

Reusable

Domain adapter proven on a stub

Submitted by **Rahul** Date **18 September 2026** Program **IIT Patna — Generative AI & Agentic AI for Developers Program**
GitHub github.com/rahulramachandran-labs/reconmind Live reconmind-labs.vercel.app

Thank you